

UNIVERSIDAD ICESI

Facultad de Ingeniería, Diseño y Ciencias Aplicadas
Departamento de Computación y Sistemas Inteligentes

TAREA EVALUATIVA N.º 1

INTERPRETACIÓN DE PRINCIPIOS Y ASPECTOS CLAVE DE DISEÑO

Caso de estudio: plataforma ApexStore

Ingeniería de Software IV (09775 - TIC) – NRC 10865

Unidad 1: Diseño y Arquitectura de Software (SWEBOK V4, capítulos 2 y 3)

Profesores: Alejandro Muñoz Bravo y Alejandro Peñaranda Agudelo

Integrantes:

Danna Isabel Zambrano – Código: A00404131

Miguel Perez – Código: A00407054

Santiago de Cali, 2026-2

Contexto de ApexStore

ApexStore Technologies es una empresa de comercio electrónico que opera en cinco países. En eventos como CyberMonday o Black Friday el tráfico se multiplica por 100 y el checkout supera las 8.000 peticiones por segundo. El equipo de ingeniería propuso la arquitectura de la Figura 1, repartida en cuatro nodos físicos: clientes, backend, pasarelas de pago y base de datos.

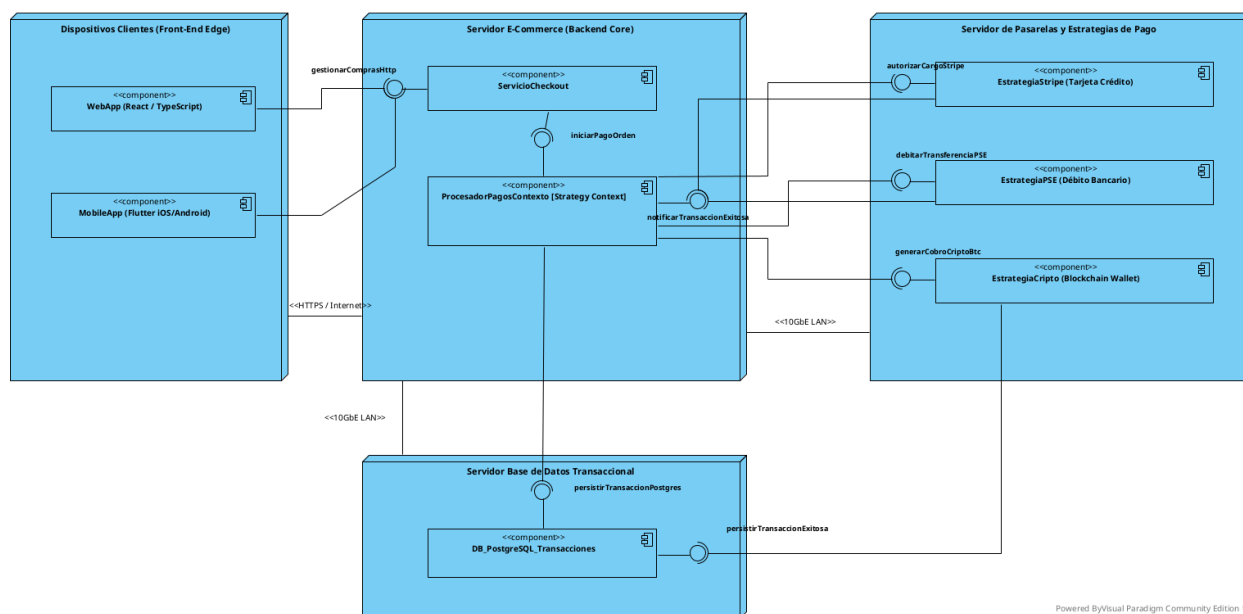


Figura 1. Diagrama de despliegue y componentes de ApexStore (enunciado).

El enunciado define cuatro Requerimientos Arquitectónicamente Significativos (RAS), que usamos como referencia en todo el informe:

- **RAS-01 (Disponibilidad y resiliencia en pagos):** el flujo de compra debe operar de forma asincrónica desacoplada; la lentitud o caída de pasarelas bancarias no debe congelar la atención de clientes (disponibilidad $\geq 99,9\%$).
- **RAS-02 (Rendimiento y latencia de red interna):** la orquestación interna de una transacción de pago entre el backend y los nodos especializados no debe superar 250 ms en el percentil 95.
- **RAS-03 (Consistencia e integridad ACID):** no se admiten pagos huérfanos, doble cobro ni inconsistencias contables; cada transacción se persiste atómica y auditadamente.
- **RAS-04 (Extensibilidad y mantenibilidad):** se deben poder agregar nuevos medios de pago con mínimo impacto, preservando el ocultamiento de información.

Punto 1: Principios de Diseño y Análisis Crítico

Para analizar el diagrama usamos los principios de diseño del SWEBOK V4 (capítulo 2) vistos en clase: abstracción, separación de preocupaciones, alta cohesión, bajo acoplamiento, ocultamiento de información, uniformidad, completitud y verificabilidad. Seleccionamos **dos**

buenos usos y tres malos usos. En cada uno mostramos el recorte del diagrama del enunciado: **en verde lo que está bien y en rojo lo que está mal.** La última columna indica qué requerimiento (RAS) se ve afectado.

#	Diagrama	Principio	Clasificación	RAS afectado
1	WebApp y MobileApp solo requieren gestionarComprasHttp de ServicioCheckout	Ocultamiento de información; bajo acoplamiento; separación de preocupaciones	Buen uso	RAS-04
2	Sistema dividido en cuatro nodos, con la base de datos en un nodo propio	Modularidad (descomposición); separación de preocupaciones; alta cohesión	Buen uso	RAS-01, RAS-02
3	Las tres estrategias exponen interfaces distintas y no hay una interfaz Strategy común	Uniformidad; ocultamiento de información; bajo acoplamiento	Mal uso	RAS-04
4	Ciclo de dependencias por notificarTransaccionExitosa	Bajo acoplamiento; verificabilidad; completitud	Mal uso	RAS-03, RAS-04
5	EstrategiaCripto escribe directamente en la base de datos	Separación de preocupaciones; alta cohesión; ocultamiento de información; uniformidad	Mal uso	RAS-03, RAS-04

1.1 Buen uso: fachada de compras gestionarComprasHttp

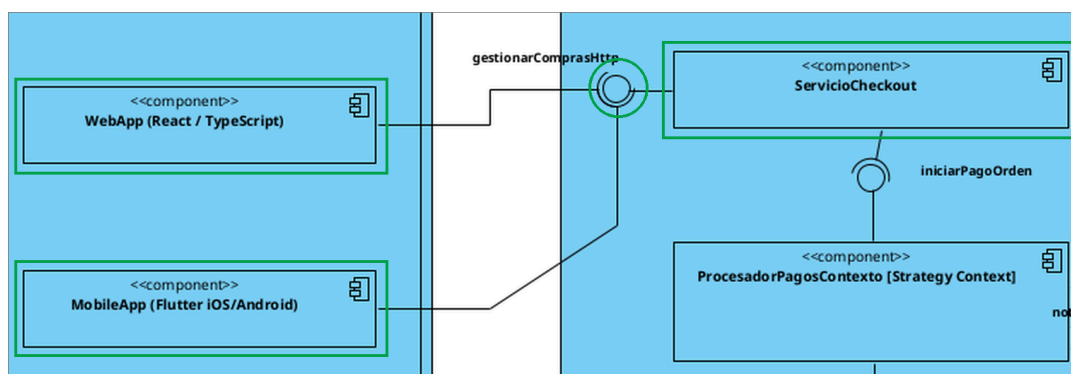


Figura 1.1. Los clientes solo se conectan a la interfaz de ServicioCheckout.

a) Evidencia en el diagrama	Como se ve en la Figura 1.1, WebApp y MobileApp, en el nodo 1, tienen un solo enchufe conectado a la bola gestionarComprasHttp, que provee ServicioCheckout en el nodo 2. Ningún cliente tiene líneas hacia ProcesadorPagosContexto, las estrategias o la base de datos. ServicioCheckout, a su vez, solo requiere iniciarPagoOrden.
Principio / clasificación	Ocultamiento de información, bajo acoplamiento y separación de preocupaciones. Buen uso.
b) Análisis del uso del principio	ServicioCheckout funciona como una fachada: los clientes conocen un solo contrato (la interfaz) y no la forma en que se procesa el pago por dentro. Así se separa la presentación (React y Flutter) de la lógica de negocio (backend). La notación también

	está bien: el ensamblaje es bola contra enchufe, sin conexiones bola con bola ni enchufe con enchufe. El único detalle es que el nombre lleva el protocolo ("Http"), que es un detalle de transporte y no debería estar en el contrato.
c) Impacto en ApexStore	Se puede cambiar lo que pasa por dentro (agregar un medio de pago, cambiar la base de datos) sin publicar una nueva versión de la app móvil ni tocar la web. La lógica del checkout no se duplica en dos lenguajes. Esto apoya directamente RAS-04: un medio de pago nuevo se agrega detrás de la fachada sin que los clientes se den cuenta. Además, solo el nodo 2 queda expuesto a los clientes, lo que reduce la superficie de ataque. El costo es que ServicioCheckout se vuelve la única puerta de entrada, así que debe poder replicarse para no convertirse en un cuello de botella en el pico de 8.000 peticiones/s.

1.2 Buen uso: cuatro nodos con la base de datos aparte

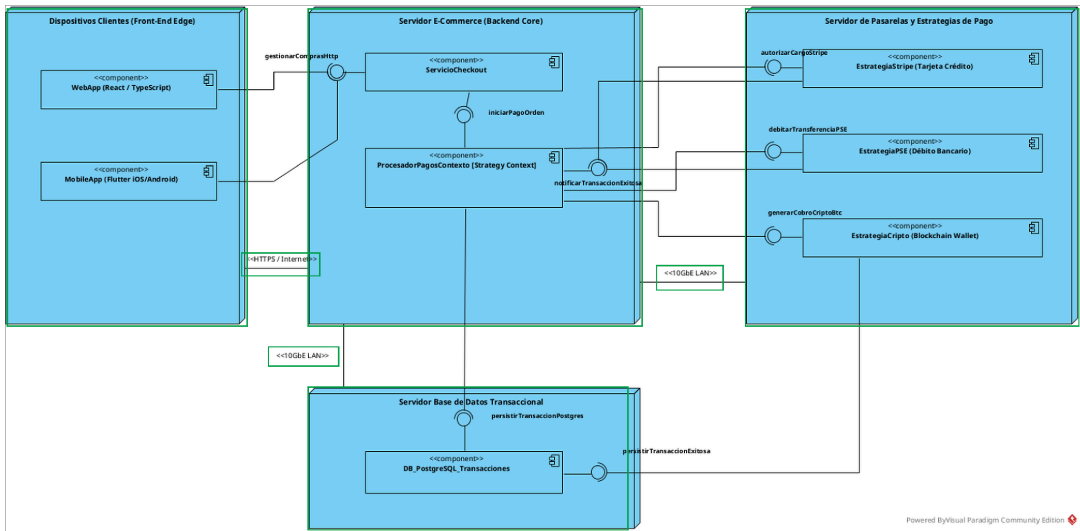


Figura 1.2. Los cuatro nodos y sus caminos de comunicación.

a) Evidencia en el diagrama	En la Figura 1.2 se ven cuatro nodos: Dispositivos Clientes, Servidor E-Commerce, Servidor de Pasarelas y Servidor Base de Datos. Hay tres caminos de comunicación, todos líneas sin flecha: nodo 1 con nodo 2 («HTTPS / Internet»), nodo 2 con nodo 3 («10GbE LAN») y nodo 2 con nodo 4 («10GbE LAN»). La base de datos está sola en su nodo.
Principio / clasificación	Modularidad (descomposición), separación de preocupaciones y alta cohesión. Buen uso.
b) Análisis del uso del principio	Cada nodo agrupa componentes con un solo propósito: presentación, orquestación del pago, integración con terceros y persistencia. Eso es alta cohesión llevada al nivel físico. Además, solo el nodo 2 tiene camino hacia los clientes; los nodos 3 y 4 quedan detrás de él. En la notación, los nodos están dibujados como cubos y los caminos como líneas sin flecha, que es lo correcto. El principio está bien aplicado, aunque la notación tiene detalles por corregir (estereotipos, entornos de ejecución y protocolos), que revisamos en el punto 1.6.
c) Impacto en ApexStore	El nodo 2 se puede escalar por separado para el pico de 8.000 peticiones/s. Si una pasarela se pone lenta, el problema queda contenido en el nodo 3 y no tumba la atención de clientes (RAS-01). La base de datos no compite por recursos con el backend, lo que ayuda a cumplir los 250 ms de RAS-02, y no es alcanzable desde Internet, lo que es coherente con PCI-DSS. El costo de separar en nodos es que

cada llamada entre ellos pasa por la red; por eso es importante que esos caminos sean una LAN rápida de 10GbE.

1.3 Mal uso: no hay una interfaz Strategy común para las estrategias

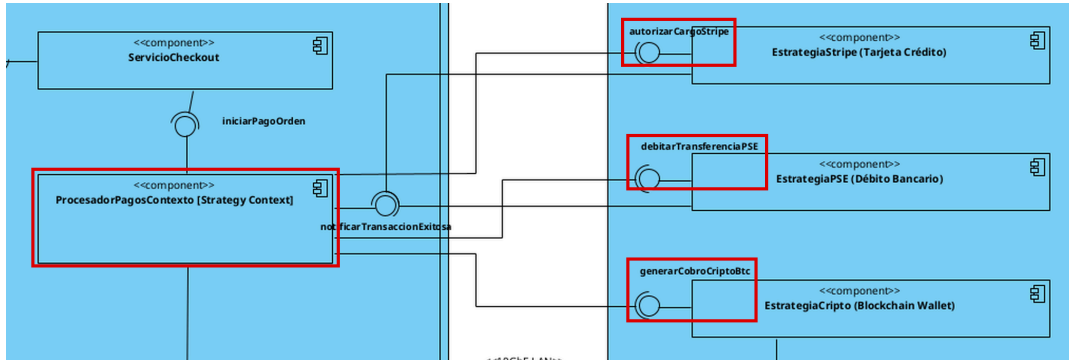


Figura 1.3. Cada estrategia expone una interfaz distinta y el contexto se conecta a las tres.

a) Evidencia en el diagrama	En el nodo 3 (Figura 1.3) cada estrategia provee una interfaz diferente: EstrategiaStripe provee <code>autorizarCargoStripe(tokenTarjeta, montoUSD, cvcSeguridad)</code> , EstrategiaPSE provee <code>debitarTransferenciaPSE(codigoBanco, tipoDoc, numCuenta, valorCOP)</code> y EstrategiaCripto provee <code>generarCobroCriptoBtc(direccionWallet, redBlockchain, satoshis)</code> . ProcesadorPagosContexto, que el diagrama marca como [Strategy Context], tiene un enchufe distinto conectado a cada una de esas tres interfaces.
Principio / clasificación	Uniformidad, ocultamiento de información y bajo acoplamiento. Mal uso.
b) Análisis del uso del principio	En el patrón Strategy el contexto debería trabajar con una sola interfaz común que todas las estrategias cumplen, sin saber cuál de ellas está usando. Aquí pasa lo contrario: cada estrategia tiene su propio contrato, con nombre, datos y moneda diferentes (dólares, pesos y satoshis). Por eso el contexto depende de tres contratos concretos y tiene que conocer los detalles de cada medio de pago, como el CVC de la tarjeta, el banco de PSE o la dirección de la billetera. Se rompe la uniformidad, porque tres piezas que hacen lo mismo (cobrar) se ven distinto; se rompe el ocultamiento de información, porque los detalles de cada pasarela quedan a la vista del contexto; y sube el acoplamiento. En la práctica, el componente se llama Strategy Context, pero no aplica el patrón.
c) Impacto en ApexStore	Agregar un medio de pago nuevo, por ejemplo una billetera digital, obliga a modificar ProcesadorPagosContexto: un enchufe nuevo, una interfaz nueva y otra rama en su lógica. Eso va en contra de RAS-04, que pide agregar medios de pago con mínimo impacto y preservando el ocultamiento de información. Además, como todos los pagos pasan por el contexto, cada cambio en él pone en riesgo los medios de pago que ya funcionan. También hace que el CVC de la tarjeta viaje hasta el contexto, cuando es un dato que no debería salir de la parte que maneja la tarjeta.

1.4 Mal uso: dependencia circular por notificarTransaccionExitosa

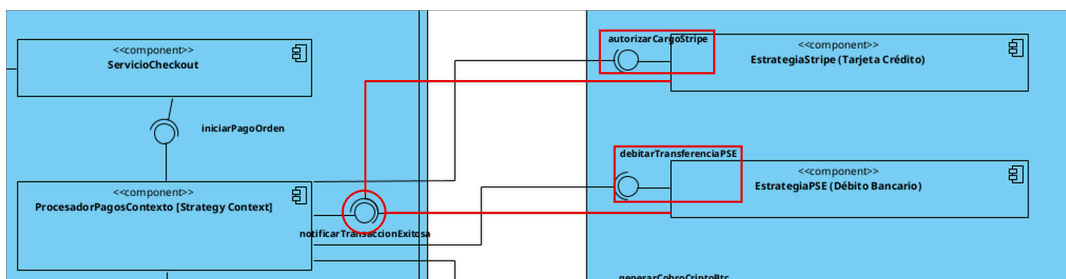


Figura 1.4. El contexto requiere a las estrategias y las estrategias requieren al contexto.

a) Evidencia en el diagrama	En la Figura 1.4, la interfaz provista notificarTransaccionExitosa está pegada a ProcesadorPagosContexto (la provee el contexto) y sus enchufes vienen de EstrategiaStripe y EstrategiaPSE. Al mismo tiempo, el contexto requiere autorizarCargoStripe y debitarTransferenciaPSE, que proveen esas mismas estrategias.
Principio / clasificación	Bajo acoplamiento, verificabilidad y completitud. Mal uso.
b) Análisis del uso del principio	Usar un callback es buena idea, porque el contexto no se queda esperando al banco. Pero como está dibujado, el contexto requiere a las estrategias y las estrategias requieren al contexto: A requiere a B y B requiere a A. En UML esto es un error semántico de componentes, la dependencia circular fuerte, que impide desacoplarlos: ninguno se puede probar, reemplazar ni desplegar sin el otro (verificabilidad). Además, el contrato está incompleto: el nombre solo cubre el éxito, no hay forma de avisar un rechazo, y EstrategiaCripto ni siquiera lo usa.
c) Impacto en ApexStore	Cualquier cambio en el contexto obliga a revisar las estrategias y al revés, lo que perjudica la mantenibilidad (RAS-04). Un pago rechazado no tiene cómo reportarse, así que la orden se puede quedar pendiente indefinidamente, lo que afecta la integridad que pide RAS-03.

1.5 Mal uso: EstrategiaCripto escribe directamente en la base de datos

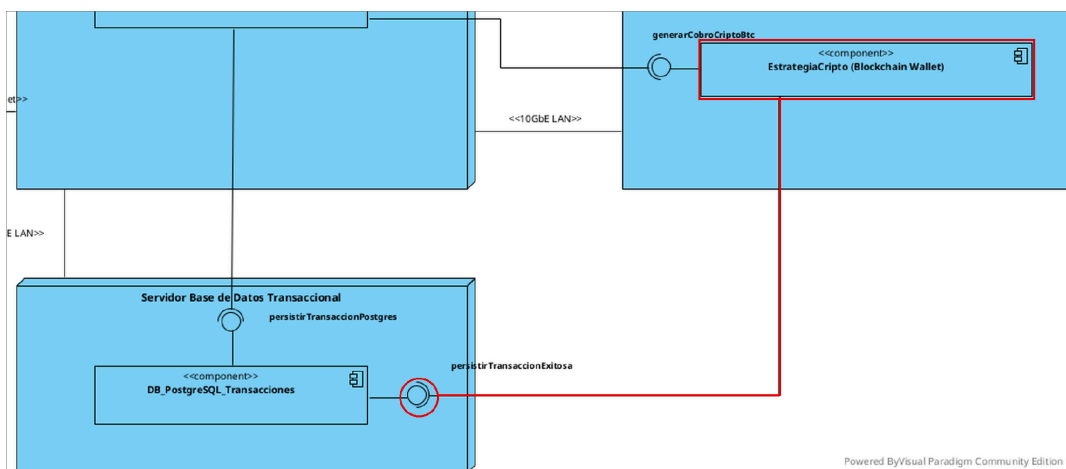


Figura 1.5. EstrategiaCripto conectada directamente a la base de datos.

a) Evidencia en el diagrama	En la Figura 1.5, EstrategiaCripto (nodo 3) tiene un enchufe conectado a la bola persistirTransaccionExitosa, que provee DB_PostgreSQL_Transacciones (nodo
-----------------------------	--

	4). No se conecta con <code>notificarTransaccionExitosa</code> como Stripe y PSE. La línea va del nodo 3 al nodo 4, pero el diagrama solo tiene caminos entre el nodo 2 y el nodo 3 y entre el nodo 2 y el nodo 4.
Principio / clasificación	Separación de preocupaciones, alta cohesión, ocultamiento de información y uniformidad. Mal uso.
b) Análisis del uso del principio	Una estrategia debería hacer una sola cosa: cobrar con su proveedor. <code>EstrategiaCripto</code> además registra el resultado, que le corresponde a quien maneja la orden. Queda un componente con baja cohesión y la persistencia repartida en dos lugares, con dos interfaces distintas sobre los mismos datos. <code>Cripto</code> se salta al componente dueño de la transacción y llega directo a los datos; eso viola el encapsulamiento, porque un componente accede a lo interno de otro. Tampoco hay uniformidad: dos estrategias reportan por callback y la tercera de otra forma, lo que empeora el problema del punto 1.3. Además, hay un error semántico de despliegue: dos nodos que se comunican sin un camino de comunicación entre ellos.
c) Impacto en ApexStore	El contexto no se entera del pago con <code>cripto</code> : la orden puede quedar pendiente con el dinero ya recibido o registrarse dos veces, justo lo que prohíbe RAS-03. Y para que la escritura directa funcionara, el nodo 3 necesitaría credenciales de escritura en la base de datos y una conexión directa al nodo 4, lo que rompe la separación de la red.

1.6 Errores e imprecisiones de notación en el diagrama de despliegue

Además de los cinco puntos anteriores, revisamos el diagrama como una especificación UML 2.5 (ISO/IEC 19505) y no como un dibujo. Hay que aclarar que UML permite algunas cosas que aparecen aquí, como un nodo sin estereotipo o un componente dibujado dentro de un nodo; por eso no las llamamos errores de sintaxis, sino **imprecisiones**: el diagrama no está mal escrito, pero queda incompleto para quien tenga que desplegar el sistema. Otras sí son **errores semánticos**, porque el diagrama dice algo que no puede pasar en la realidad.

Nodos sin estereotipo (imprecisión). Una caja que no dice si es «device» o «executionEnvironment» no permite saber si es hardware o el software que ejecuta el código. Pasa en los cuatro nodos (Figura 1.6).



Figura 1.6. Encabezados de los nodos sin estereotipo.

Nodos sin entorno de ejecución (incompletitud). Los componentes están dibujados directo dentro de cada cubo y no hay ningún «executionEnvironment» (Figura 1.7): no aparece el navegador donde corre la WebApp, el runtime de Flutter de la MobileApp, el entorno donde corren el backend y las pasarelas, ni el motor de PostgreSQL. Un diagrama de despliegue debería separar el hardware («device»), el software que ejecuta el código («executionEnvironment») y lo que se despliega («artifact»). Sin eso no se sabe qué plataforma necesita cada nodo, que es justo lo que hay que saber para montarlo, y tampoco queda dónde configurar cosas que importan en el pico de 8.000 peticiones/s, como la memoria o el tamaño del thread pool del servidor.

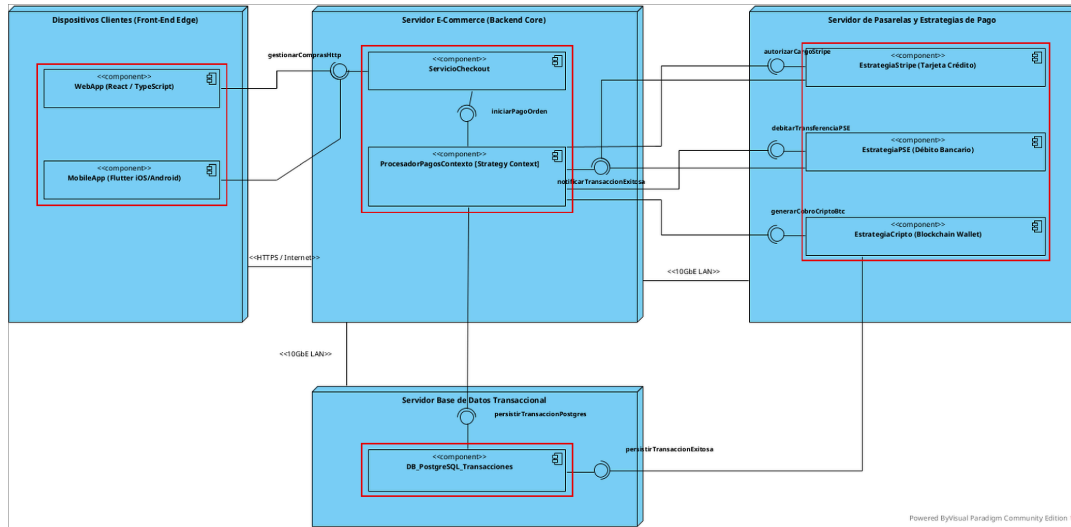


Figura 1.7. Componentes dibujados directamente en los nodos, sin entorno de ejecución.

Componentes sin artefacto (incompletitud). Los componentes no se copian tal cual al servidor: se empaquetan en un «artefacto» (por ejemplo, backend.jar) que los manifiesta («manifest»), y ese artefacto es lo que se despliega. El diagrama no muestra ningún artefacto (Figura 1.8).

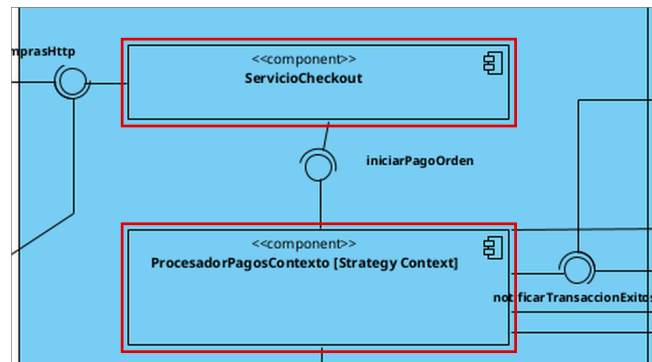


Figura 1.8. Componentes puestos directamente en el Servidor E-Commerce.

Nodo de clientes (error semántico). Un nodo debe corresponder a un recurso físico real, y "Dispositivos Clientes" mete en una sola caja un computador con navegador y un smartphone, que son dos equipos distintos (Figura 1.9).

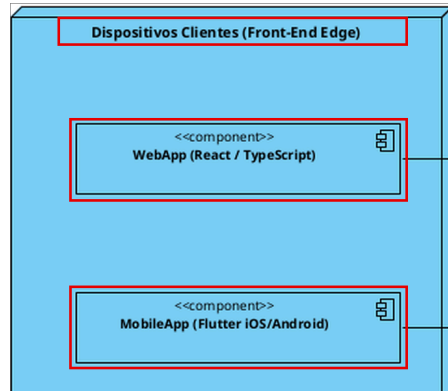


Figura 1.9. Nodo de clientes con dos dispositivos distintos.

Protocolo omitido (error semántico). Un camino entre dos equipos distintos debe decir con qué protocolo se comunican (TCP/IP, HTTP/S, ICE). «HTTPS / Internet» sí lo indica, pero «10GbE LAN» solo describe el medio físico (Figura 1.10).

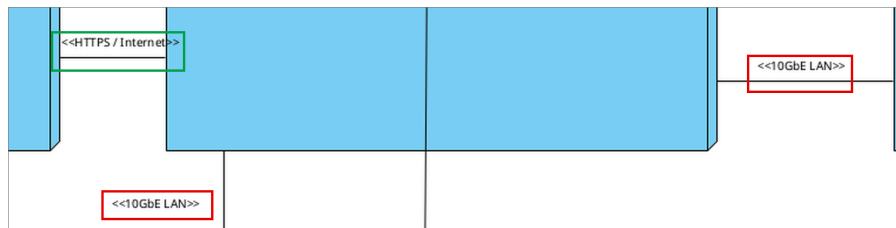


Figura 1.10. Etiquetas de los caminos de comunicación.

Pasarelas externas sin camino (error semántico). Las estrategias cobran con Stripe, el banco (PSE) y la red blockchain, que están fuera de ApexStore, pero el diagrama no muestra esos sistemas ni un camino que salga del nodo 3 hacia ellos (Figura 1.11).

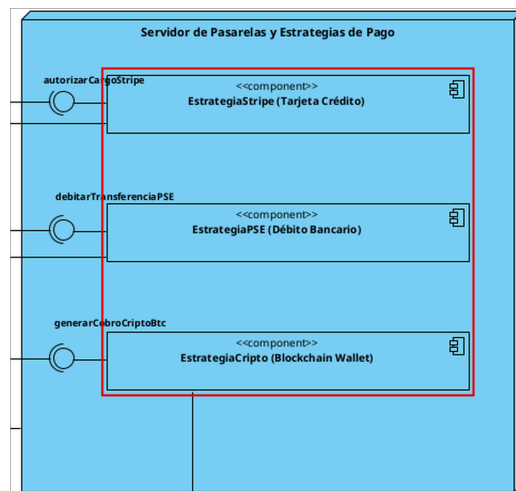


Figura 1.11. Nodo 3 sin camino hacia las pasarelas externas.

La base de datos como componente (imprecisión). UML permite dibujar una base de datos como componente, pero aquí mezcla tres cosas distintas: el componente lógico que guarda las

transacciones, PostgreSQL, que es el entorno que ejecuta la base de datos (DBMS), y la base de datos en sí, que es lo que se despliega en él («artifact») (Figura 1.12).

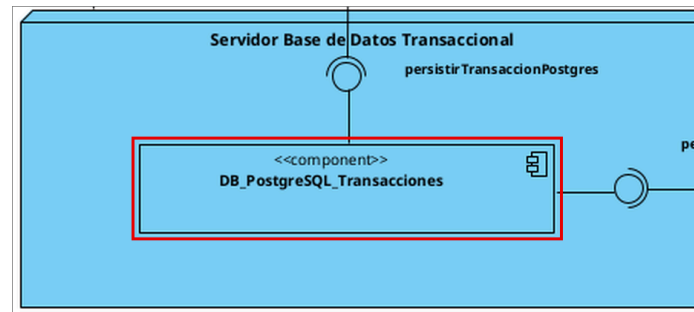


Figura 1.12. DB_PostgreSQL_Transacciones dibujada como «component».

Lo que sí está bien en la notación. Los nodos están dibujados como cubos, los componentes usan «component» y su ícono de UML 2.x, todos los ensamblajes son bola contra enchufe (no hay bola con bola ni enchufe con enchufe) y los caminos de comunicación son líneas sin flecha (Figuras 1.1 y 1.2).

Punto 2: Análisis Profundo de los Aspectos Clave de Diseño

Respondemos las seis preguntas del enunciado con base en los aspectos clave de diseño del SWEBOK V4 (capítulo 2) vistos en la Sesión 3. Cada respuesta se apoya en el diagrama original (Figura 1).

2.1 Dimensión 1: Interacción y Presentación

¿Por qué los clientes front-end interactúan exclusivamente con la interfaz gestionarComprasHttp provista por ServicioCheckout, en lugar de conectarse directamente con el procesador de pagos o la base de datos?

R// Porque así cada parte se encarga solo de lo suyo. WebApp y MobileApp solo muestran información y capturan lo que el usuario quiere hacer; las reglas de negocio (validar la compra, crear la orden, elegir la estrategia y registrar la transacción) le pertenecen al backend. Si los clientes llamaran directamente a ProcesadorPagosContexto, la lógica del checkout tendría que repetirse en React y en Flutter, y cualquier diferencia entre las dos versiones terminaría en un error contable.

También es un tema de encapsulamiento. ProcesadorPagosContexto, las estrategias y PostgreSQL son detalles internos; si el cliente los conoce, cualquier cambio interno se convierte en un cambio del contrato público. Con la fachada, los front-ends dependen de una sola interfaz y el backend puede agregar estrategias, cambiar de base de datos o reorganizar nodos sin obligar a actualizar la app móvil, que depende de los tiempos de las tiendas de aplicaciones y de que el usuario la actualice.

Por último, está la seguridad y el control de la carga. Los clientes corren en dispositivos que ApexStore no controla y se conectan por Internet. Conectarlos directo a la base de datos obligaría a poner credenciales dentro de la app, donde se pueden extraer, y a exponer el nodo 4 a Internet. Con la fachada, lo único expuesto es una interfaz HTTPS donde se autentica al

usuario y se valida cada solicitud. Y en un pico de 8.000 peticiones/s, tener un solo punto de entrada permite limitar las peticiones y replicar ese componente sin tocar el resto.

Esta decisión apoya RAS-04, porque los medios de pago cambian sin afectar a los clientes, y RAS-01, porque la carga se controla en un solo lugar. El precio es que toda compra pasa por ServicioCheckout: si ese componente falla o se satura, nadie puede comprar. Por eso la fachada debe ser liviana (solo recibir, validar y delegar) y poder replicarse.

2.2 Dimensión 2: Concurrencia y Rendimiento

En un pico de 8.000 peticiones/s, ¿por qué la combinación de despacho síncrono inicial con confirmación asíncrona vía webhook/callback protege el thread pool del servidor frente a retardos bancarios de 15 segundos?

R// La afirmación es correcta, siempre que el despacho síncrono inicial solo entregue el cobro a la pasarela y espere un "recibido", y que la respuesta del banco llegue después por el callback. Lo justificamos con los aspectos clave de diseño vistos en clase.

Desde la concurrencia, el servidor E-Commerce atiende cada petición con un hilo del thread pool, y ese pool tiene un tamaño fijo. Mientras un hilo espera una respuesta, no puede atender a nadie más. Si cada compra dejara su hilo esperando los 15 segundos del banco, en ese tiempo seguirían llegando miles de compras nuevas cada segundo y ningún hilo se liberaría. El pool se agotaría casi de inmediato y el servidor dejaría de responder, incluso a las compras con medios de pago que sí funcionan.

El despacho síncrono inicial evita ese bloqueo porque es corto: el contexto le entrega el cobro a la estrategia del nodo 3 por la LAN de 10GbE y solo espera la confirmación de que lo recibió, que no depende del banco. Con ese acuse, el servidor le responde al cliente que la orden quedó PENDIENTE y el hilo vuelve al pool para atender la siguiente compra.

La confirmación asíncrona corresponde a la gestión de eventos: es el desacoplamiento temporal, en el que quien emite el evento no se queda esperando la respuesta inmediata del receptor. Cuando el banco termina, la pasarela avisa por el webhook/callback (notificarTransaccionExitosa en el diagrama) y ese aviso ocupa un hilo solo lo que tarda en actualizar la orden, no los 15 segundos.

En rendimiento, la latencia que percibe el cliente pasa a ser la del despacho y no la del banco, y la cantidad de compras atendidas por segundo depende de la capacidad del nodo 2 y no de la lentitud de un tercero. Por eso los 250 ms en P95 de RAS-02 solo se pueden cumplir si la espera del banco queda por fuera del camino síncrono. Y en tolerancia a fallos, como ningún hilo queda amarrado al banco, un banco lento no le quita hilos a las demás compras y se evita la falla en cascada, que es lo que pide RAS-01.

Ahora bien, en el diagrama del enunciado esto se cumple solo en parte. El enunciado dice que el contexto "despacha los cobros síncronos a las pasarelas", pero no aclara qué espera esa llamada: si espera a que la estrategia termine con el banco, el hilo del nodo 2 queda bloqueado igual, y tampoco se define un tiempo máximo de espera. Además, solo Stripe y PSE confirman por notificarTransaccionExitosa;

EstrategiaCripto escribe directo en la base de datos (punto 1.5), así que el contexto no recibe aviso. Y el callback solo cubre el éxito: si el banco rechaza el pago no hay evento y la orden se queda pendiente.

Este esquema también tiene un costo: el cliente no recibe la respuesta final en el momento, sino una orden PENDIENTE, y el sistema se vuelve más complejo porque debe manejar estados intermedios, avisos que llegan tarde o repetidos y la confirmación posterior al cliente. Aun así, para ApexStore vale la pena, porque la alternativa (esperar al banco) tumba el servidor en el pico.

En conclusión, el esquema de despacho síncrono corto con confirmación asíncrona sí protege el thread pool, porque ningún hilo espera los 15 segundos del banco. Para que se cumpla de verdad en ApexStore, el despacho debe tener un tiempo máximo de espera y todas las estrategias, incluida Cripto, deben confirmar por el mismo callback, tanto si el pago se aprueba como si se rechaza. Esto se corrige en el Punto 3.

2.3 Dimensión 3: Tolerancia a Fallos y Manejo de Errores

Si la pasarela PSE no responde o la red blockchain se congestiona, ¿cómo debe reaccionar el contexto para que el fallo de una estrategia no colapse las transacciones de los otros métodos de pago?

R// Lo primero es aislar el fallo. Cada estrategia debe tener su propio tiempo máximo de espera y su propio circuit breaker (un interruptor que, si un medio de pago falla varias veces seguidas, deja de enviarle cobros por un rato y responde de inmediato que no está disponible), y el contexto nunca debe dejar un hilo bloqueado esperando a una estrategia. Así, un problema en PSE no consume los recursos que necesitan Stripe o Cripto, y al cliente se le puede ofrecer pagar con otro medio.

Además, hay que tener en cuenta cómo funciona cada medio. En PSE el cliente es redirigido a la página de su banco para aprobar el débito; puede tardar, abandonar el proceso o el banco puede no responder, y el resultado llega mucho después o no llega. En blockchain, cuando la red está congestionada, las confirmaciones de una transacción pueden tardar minutos o más. En los dos casos el pago no ha fallado necesariamente; simplemente no se sabe todavía. Por eso el contexto no debe marcarlo como FALLIDO por un tiempo de espera corto, sino dejarlo en un estado intermedio, **EN_VERIFICACION**, y consultar o esperar el resultado real. Cuando llega, la orden pasa a APROBADO o RECHAZADO. Esto evita el caso de un pago que el sistema dio por fallido pero que el banco sí cobró (pago huérfano).

En el manejo de excepciones, todo error de comunicación con una pasarela se convierte en un estado de negocio guardado y auditado (EN_VERIFICACION o RECHAZADO), nunca en una orden "perdida". Y los reintentos deben ser controlados: solo se reintentan operaciones que se pueden repetir sin efecto adicional, como la entrega del callback, que se puede reintentar hasta 3 veces porque aplicarla dos veces no cambia nada. El cobro en sí no se reintenta automáticamente.

Por último, cada intento de pago debe llevar una **clave de idempotencia** asociada a la orden. Si el cliente, al ver que PSE no responde, intenta pagar de nuevo o con otro medio, el sistema

revisa primero si esa orden ya tiene un pago en EN_VERIFICACION o APROBADO, y así no se genera un doble cobro (RAS-03). Con estos elementos se cumple lo que pide la pregunta sin agregar componentes más pesados, como colas de mensajes. El costo es que algunos clientes de PSE o cripto verán su orden "en verificación" por un rato en vez de una respuesta inmediata, pero es preferible a perder un pago o cobrar dos veces (RAS-01 y RAS-03).

2.4 Dimensión 4: Gestión de Eventos y Control de Flujo

Compare el flujo de control entre el callback que recibe ProcesadorPagosContexto y la inserción directa que hace EstrategiaCripto. Si Cripto escribe directo a la base de datos, ¿cómo se entera el contexto para actualizar la orden y enviar la confirmación al cliente?

R// Con Stripe y PSE el flujo es este: el contexto despacha el cobro a la estrategia, la pasarela lo procesa con el banco y, cuando termina, la estrategia invoca `notificarTransaccionExitosa`. El contexto recibe ese evento y actualiza la transacción y la orden en la base de datos por `persistirTransaccionPostgres`. Es decir, el contexto controla el flujo de principio a fin.

Con Cripto el flujo se rompe en el último paso. El contexto despacha el cobro, la red blockchain confirma la transacción y `EstrategiaCripto` invoca `persistirTransaccionExitosa`, que va directo a la base de datos. La base de datos recibe el dato, pero el contexto no recibe nada. Así que, con el diagrama tal como está, **el contexto no se entera**. PostgreSQL sí tiene un mecanismo para avisar cambios (LISTEN/NOTIFY), pero no aparece en el diseño y usarlo amarraría el contexto a la base de datos. La otra salida sería que el contexto consultara la base de datos cada cierto tiempo (polling), lo cual tampoco está en el diseño y además genera carga y retraso. Mientras tanto, nadie actualiza la orden de forma controlada: o se queda PENDIENTE para siempre, o `EstrategiaCripto` tendría que conocer también el modelo de órdenes, lo que aumenta el acoplamiento.

Hay que aclarar algo sobre la confirmación al cliente: el diagrama no la resuelve para **ningún** medio de pago, ni siquiera para Stripe y PSE. `ServicioCheckout` solo se relaciona con el contexto requiriendo `iniciarPagoOrden`; no existe una interfaz por la que el contexto le avise a `ServicioCheckout` que el pago terminó, ni una por la que el cliente pueda consultar el estado de su orden. Por eso, en todos los casos, el cliente se queda con la orden PENDIENTE. El Punto 3 debe definir cómo se confirma al cliente, ya sea con una consulta de estado o con un aviso desde el servidor.

La inconsistencia arquitectónica es que el sistema tiene dos flujos de control para el mismo evento ("el pago terminó"), y en uno de ellos el componente que controla el flujo (el contexto) queda por fuera. Eso produce pagos huérfanos, posibles registros duplicados si el contexto intenta guardar el resultado por su lado, y rompe la uniformidad del patrón Strategy. A esto se suma el problema del punto 1.4: como el contexto es quien provee `notificarTransaccionExitosa`, el contexto requiere a las estrategias y las estrategias requieren al contexto, una dependencia circular fuerte que impide que los dos componentes evolucionen o se prueben por separado. En resumen, el flujo de Cripto va en contra de RAS-03 (pagos huérfanos o duplicados) y de RAS-04 (cada medio de pago se comporta distinto), y la falta de confirmación al cliente afecta a todos los medios de pago.

2.5 Dimensión 5: Persistencia de Datos e Integridad Transaccional

¿Por qué es imperativo que DB PostgreSQL Transacciones garantice propiedades ACID para los cobros, en contraste con un modelo de consistencia eventual (BASE)?

R// Porque en un cobro no se puede aceptar ni un momento de datos inconsistentes, y eso es justo lo que garantiza ACID. Con la **atomicidad**, registrar el pago aprobado y cambiar la orden a APROBADO son un solo paso: o se hacen las dos cosas o ninguna. Si el servidor se cae en la mitad, no puede quedar un pago registrado sin orden aprobada (pago huérfano) ni una orden aprobada sin pago. Con la **consistencia**, cada transacción lleva la base de datos de un estado válido a otro respetando sus reglas: una clave de idempotencia única para que un reintento no cree un segundo cobro, que todo pago esté ligado a una orden, que los montos sean positivos y que los estados solo cambien en el orden permitido (por ejemplo, APROBADO no vuelve a PENDIENTE).

El **aislamiento** es clave con 8.000 peticiones/s, porque pueden llegar al mismo tiempo el callback de un pago y un reintento del cliente sobre la misma orden. Sin aislamiento, ambos podrían leer PENDIENTE y aplicar el resultado dos veces, lo que termina en doble cobro o doble registro. Y la **durabilidad** asegura que, si el banco aprobó y ApexStore ya le confirmó al cliente, ese registro siga existiendo aunque el nodo 4 se reinicie; en PostgreSQL esto lo da el registro de escritura anticipada (WAL).

BASE (Basically Available, Soft state, Eventual consistency) acepta que durante un tiempo distintas copias de los datos muestren cosas diferentes, a cambio de más disponibilidad y escalabilidad. Es una buena opción para partes de ApexStore donde un dato desactualizado por unos segundos no causa daño, como el catálogo, las recomendaciones, las reseñas o el buscador. El precio de ACID es que limita la escalabilidad: las transacciones bloquean los registros que están usando y una sola base de datos no crece tan fácil como un sistema BASE. Por eso conviene usar ACID solo donde es indispensable. Y en el núcleo contable no hay alternativa: si dos copias no coinciden por un momento sobre si un pago existe, se puede despachar un producto sin cobrar, cobrar dos veces o no poder cuadrar las cuentas con el banco. Además, un pago tiene consecuencias externas que no se pueden deshacer, porque el dinero ya se movió. RAS-03 lo dice claramente: "bajo ninguna circunstancia" pagos huérfanos, doble cobro o inconsistencias contables, con persistencia atómica y auditada.

Hay un matiz importante: la relación entre ApexStore y el banco sí es eventualmente consistente, porque el resultado llega después por callback. Justamente por eso el registro interno debe ser estricto: guardar los estados del pago con cambios atómicos, operaciones idempotentes y auditoría es lo que permite manejar la incertidumbre del mundo externo sin dañar los datos propios.

2.6 Dimensión 6: Distribución de Componentes

Justifique la topología física en 4 nodos de ejecución interconectados por HTTPS / Internet y 10GbE LAN bajo el estándar de seguridad de datos financieros (PCI-DSS).

R// Cada nodo tiene una razón de ser. El nodo 1 (Dispositivos Clientes) son equipos del usuario, que ApexStore no controla y en los que no se puede confiar; por eso solo presentan y

capturan datos, y son los únicos que llegan por Internet. El nodo 2 (Backend Core) es la única puerta pública: concentra la orquestación y las reglas de negocio, y se puede escalar por separado para aguantar el pico de 8.000 peticiones/s. El nodo 3 (Pasarelas y Estrategias) aísla la integración con terceros financieros: si una pasarela se pone lenta o consume recursos, el daño queda contenido en ese nodo (RAS-01), y al separarlo se limita qué equipos manejan datos de pago. El nodo 4 (Base de Datos) es el activo más crítico; tenerlo dedicado evita que compita por CPU y disco con el backend, permite protegerlo y respaldarlo por aparte, y lo deja fuera de Internet.

Entre el nodo 1 y el nodo 2 la comunicación atraviesa una red pública, por eso se usa HTTPS, que cifra los datos, protege su integridad y autentica al servidor. Entre los nodos 2, 3 y 4 la comunicación va por una LAN privada de 10 Gb/s, que da baja latencia y mucho ancho de banda (RAS-02) y, sobre todo, permite **segmentar** la red. Hay que precisar cómo queda cada nodo interno: el nodo 4 no tiene ningún acceso desde ni hacia Internet, mientras que el nodo 3 no acepta conexiones que entren desde Internet, pero sí necesita salida controlada hacia Stripe, PSE y la red blockchain para poder cobrar. Así, un atacante que comprometa un cliente solo ve la interfaz de la fachada.

Esta topología es coherente con varios requisitos de PCI-DSS. El requisito 1 pide controles de seguridad de red; separar la zona que procesa y guarda datos de pago del resto y de Internet reduce el alcance del entorno de datos de tarjetahabientes. El requisito 3 pide proteger los datos de cuenta almacenados, y en particular no guardar el código de verificación de la tarjeta (CVC) después de autorizar el pago; por eso conviene tokenizar la tarjeta en el cliente para que el CVC no llegue al backend, a diferencia del diagrama original, donde `cvcSeguridad` pasa por el contexto. El requisito 4 pide cifrar los datos de tarjeta cuando viajan por redes públicas, y eso lo cubre HTTPS entre clientes y backend. Y el requisito 10 pide registrar y monitorear los accesos a los datos de tarjetahabientes, lo que coincide con la persistencia auditada de RAS-03.

Separar en cuatro nodos tiene un costo: cada pago hace varios saltos de red (del nodo 2 al 3 y del 2 al 4), lo que suma latencia y más puntos que pueden fallar. La LAN de 10GbE reduce ese costo y permite mantener la orquestación interna dentro de los 250 ms de RAS-02. Además, la topología tiene una debilidad frente al 99,9 % de disponibilidad de RAS-01: hay un solo nodo de base de datos y un solo nodo de pasarelas, así que cada uno es un punto único de falla. Si cualquiera de los dos se cae, se caen todos los pagos. Para cumplir ese requerimiento conviene tener una réplica de la base de datos con conmutación automática (failover) y más de una instancia del nodo de pasarelas.